Processamento de Linguagem Natural

Resolução dos Exercícios TC.10 e PC.10

Disciplina: Processamento de Linguagem Natural

Professor: Fabrício Galende Marques de Carvalho

Aluno(a): (preencher)

São José dos Campos, 24 de agosto de 2026

Sumário

1. TC.10 - Emojis e símbolos em sistemas de PLN
2. PC.10 - Assistente virtual baseado em tags POS
3. Conclusão
4. Referências

TC.10

Emojis e símbolos não formais em sistemas de PLN

TC.10 - Enunciado

"Por que a utilização de emojis ou outros símbolos não presentes na linguagem textual formal podem dificultar a operação de um sistema de PLN? Cite um exemplo prático de sentença textual comumente presente em diálogos que faz uso de emojis e ilustre qual a dificuldade pode ser gerada ao sistema de PLN."

Trata-se de um exercício conceitual: a resposta é construída em torno dos fundamentos de léxico, tokenização e etiquetagem morfossintática (POS) apresentados no material da disciplina.

TC.10 - Conceitos envolvidos

Léxico e vocabulário: conjunto de palavras e expressões reconhecidas por um sistema de PLN, definido a partir do corpus utilizado no treinamento.

Tokenização: segmentação de um texto em unidades menores (tokens), como palavras e sinais de pontuação.

POS tagging (etiquetagem morfossintática): atribuição da classe gramatical (substantivo, verbo, adjetivo etc.) a cada token.

Emojis não pertencem à linguagem textual formal: são símbolos pictográficos Unicode, sem entrada no léxico da língua e sem papel morfossintático definido.

TC.10 - Exemplo prático

Frase de exemplo (comum em avaliações de produto e conversas em redes sociais):

"Adorei o produto 😍👏, mas a entrega atrasou muito 😡"

Resultado obtido com spaCy (modelo pt_core_news_sm):

```
Token       POS      Lema
---------   ------   ---------
Adorei      VERB     Adorei
produto     NOUN     produto
😍           ADJ      😍           <- classe gramatical sem sentido
👏           NOUN     👏           <- classe gramatical sem sentido
atrasou     VERB     atrasar
😡           VERB     😡           <- classe gramatical sem sentido

NLTK (word_tokenize) agrupa os dois primeiros emojis em um único token: "😍👏"
```

TC.10 - Por que é um problema, e como resolver

O problema

Classificação POS sem sentido: o modelo nunca viu emojis no treinamento.

Tokenizadores divergem entre si, prejudicando a reprodutibilidade da pipeline.

Perda de carga emocional: emojis carregam sentimento relevante para tarefas como análise de sentimentos; ignorá-los descarta um sinal importante.

A solução

Etapa dedicada de pré-processamento, antes da tokenização/POS.

Conversão do emoji para um token textual equivalente (ex.: 😍 -> "carinha_apaixonada"), usando bibliotecas como emoji ou demoji.

Alternativa: mapear o emoji direto para um rótulo de sentimento (positivo/negativo/neutro), preservando a informação emocional na pipeline.

PC.10

Assistente virtual baseado em tags POS

PC.10 - Enunciado

"Fazendo uso de tags do tipo POS, desenvolva um assistente virtual simples que identifique se o usuário está fazendo uma pergunta ou se é afirmação ou comando."

Dica do enunciado: observar a primeira palavra da frase e o seu POS.

Ferramentas: Python 3, biblioteca spaCy e o modelo pt_core_news_sm, que fornece POS e morfologia (incluindo o modo verbal) para cada token.

PC.10 - Estratégia adotada

Comando: a primeira palavra é um verbo. No português, o imperativo afirmativo (você/ele) é morfologicamente idêntico ao presente do subjuntivo - por isso, tanto Mood=Imp quanto Mood=Sub são aceitos como indício de comando.

Pergunta: a primeira palavra é um pronome ou advérbio interrogativo (quem, onde, como, quando, qual...) OU a frase termina em "?" (necessário para perguntas do tipo sim/não, sem marca morfológica na primeira palavra).

Afirmação: caso nenhuma das regras anteriores seja satisfeita.

Pré-processamento: a primeira letra da frase é colocada em minúscula antes do POS tagging, reduzindo o viés de maiúscula inicial (comum em modelos estatísticos pequenos, que tendem a marcar palavras capitalizadas como nomes próprios).

PC.10 - Código-fonte (função principal)

```
def identificar_tipo_frase(frase: str) -> str:
    frase = frase.strip()
    frase_norm = frase[0].lower() + frase[1:]
    doc = nlp(frase_norm)
    primeiro = doc[0]
    pos = primeiro.pos_
    modo = primeiro.morph.get("Mood")

    if pos in ("PRON","DET","ADV","SCONJ") and \
       primeiro.text.lower() in PALAVRAS_INTERROGATIVAS:
        return "Pergunta"
    if pos in ("VERB","AUX") and modo and modo[0] in ("Imp","Sub"):
        return "Comando"
    if frase.endswith("?"):
        return "Pergunta"
    return "Afirmação"
```

Código completo, com células executáveis, disponível em TC10_PC10_PLN.ipynb.

PC.10 - Resultados dos testes

```
Frase                                  Classificação
-------------------------------------------------------
Você fechou a porta?                   Pergunta
Feche a porta, por favor.              Comando
O gato está dormindo no sofá.          Afirmação
Quem chamou você ontem?                Pergunta
Envie o relatório até amanhã.          Comando
Ela chegou cedo para a reunião.        Afirmação
Onde fica a estação mais próxima?      Pergunta
Abra a porta.                          Comando
```

As 8 frases de teste cobrem os três tipos de sentença (pergunta com e sem pronome interrogativo, comando e afirmação) e foram todas classificadas corretamente pelas regras baseadas em POS.

PC.10 - Limitações observadas

O modelo pt_core_news_sm é pequeno e erra a POS de verbos pouco frequentes no início da frase. Exemplo: em "Compre pão no mercado.", a palavra "compre" é rotulada como ADJ (em vez de VERB), e a frase acaba classificada erroneamente como afirmação, quando deveria ser um comando.

Perguntas do tipo sim/não dependem exclusivamente do sinal "?", pois o português não marca isso na primeira palavra da frase. Em fala transcrita sem pontuação, essa regra deixaria de funcionar.

Orações subordinadas antepostas (ex.: "Se puder, feche a porta.") não são tratadas corretamente, pois a primeira palavra não corresponde ao verbo principal da frase.

Modelos maiores (ex.: pt_core_news_lg) tendem a cometer menos erros desse tipo, ao custo de maior tempo de processamento e consumo de memória.

PC.10 - Dependências e execução

```
requirements.txt
-----------------
spacy==3.8.16
nltk==3.10.3
jupyter==1.1.1
+ modelo pt_core_news_sm (URL direta no requirements.txt)
```

1. python3 -m venv venv && source venv/bin/activate
2. pip install -r requirements.txt
3. jupyter notebook TC10_PC10_PLN.ipynb
4. Executar todas as células (Run All). A função assistente_virtual() é interativa e deve ser chamada manualmente em uma célula própria.

Conclusão

TC.10 evidenciou que símbolos fora do léxico formal, como os emojis, quebram suposições básicas dos modelos de PLN, exigindo tratamento específico em uma etapa dedicada da pipeline.

PC.10 mostrou, na prática, como tags POS - mesmo obtidas por um modelo estatístico relativamente simples - permitem construir uma heurística funcional para classificar o tipo de uma frase em português.

Em ambos os casos, a qualidade e a cobertura do modelo de linguagem utilizado impactam diretamente a confiabilidade do sistema de PLN resultante.

Referências

AGGARWAL, C. C. Machine Learning for Text. Switzerland: Springer Nature Switzerland, 2023.

FERREIRA, M.; LOPES, M. Para Conhecer Linguística Computacional. São Paulo, SP: Contexto, 2025.

SARKAR, D. Text Analytics with Python. 2nd ed. Berkeley, CA: Apress, 2019.

HONNIBAL, M. et al. spaCy: Industrial-strength Natural Language Processing in Python. Disponível em: https://spacy.io

CARVALHO, F. G. M. Notas de aula do curso Processamento de Linguagem Natural. São José dos Campos, 2025.